UNIT-VIII

FIREWALLS

A firewall is inserted between the premises network and the Internet to establish a controlled link and to erect an outer security wall or perimeter, forming a single choke point where security and audit can be imposed. A firewall:

1. Defines a single choke point that keeps unauthorized users out of the protected network, prohibits potentially vulnerable services from entering or leaving the network, and provides protection from various kinds of IP spoofing and routing attacks.
2. Provides a location for monitoring security-related events
3. Is a convenient platform for several Internet functions that are not security related, such as NAT and Internet usage audits or logs
4. A firewall can serve as the platform for IPSec to implement virtual private networks.

Design Goals of Firewalls

- All traffic from inside to outside must pass through the firewall (physically blocking all access to the local network except via the firewall)
- Only authorized traffic (defined by the local security police) will be allowed to pass
- The firewall itself is immune to penetration (use of trusted system with a secure operating system)

The 4 general techniques that firewalls use to control access and enforce the sites security policies are:

- *Service control*: Determines the types of Internet services that can be accessed, inbound or outbound
- *Direction control*: Determines the direction in which particular service requests are allowed to flow
- *User control*: Controls access to a service according to which user is attempting to access it
- *Behavior control*: Controls how particular services are used (e.g. filter e-mail)

The limitations of Firewalls are:

1. Cannot protect against attacks that bypass the firewall, eg PCs with dial-out capability to an ISP, or dial-in modem pool use.
2. Do not protect against internal threats, eg disgruntled employee or 1 who cooperates with an attacker
3. Cannot protect against the transfer of virus-infected programs or files, given wide variety of O/S & applications supported

Types of Firewalls

Firewalls are generally classified as three types: packet filters, application-level gateways, & circuit-level gateways.

Packet-filtering Router

A packet-filtering router applies a set of rules to each incoming and outgoing IP packet to forward or discard the packet. Filtering rules are based on information contained in a network packet such as src & dest IP addresses, ports, transport protocol & interface.

If there is no match to any rule, then one of two default policies are applied:

- that which is not expressly permitted is prohibited (default action is discard packet), conservative policy
- that which is not expressly prohibited is permitted (default action is forward packet), permissive policy

The default discard policy is more conservative. Initially, everything is blocked, and services must be added on a case-by-case basis. This policy is more visible to users, who are more likely to see the firewall as a hindrance. The default forward policy increases ease of use for end users but provides reduced security; the security administrator must, in essence, react to each new security threat as it becomes known. One advantage of a packet-filtering router is its simplicity. Also, packet filters typically are transparent to users and are very fast. The table gives few examples of packet-filtering rule sets. In each set, the rules are applied top to bottom.

Table 20.1 Packet-Filtering Examples

A

action	ourhost	port	theirhost	port	comment
block	*	*	SPIGOT	*	we don't trust these people
allow	OUR-GW	25	*	*	connection to our SMTP port

B

action	ourhost	port	theirhost	port	comment
block	*	*	*	*	default

C

action	ourhost	port	theirhost	port	comment
allow	*	*	*	25	connection to their SMTP port

D

action	src	port	dest	port	flags	comment
allow	{our hosts}	*	*	25		our packets to their SMTP port
allow	*	25	*	*	ACK	their replies

E

action	src	port	dest	port	flags	comment
allow	{our hosts}	*	*	*		our outgoing calls
allow	*	*	*	*	ACK	replies to our calls
allow	*	*	*	>1024		traffic to nonservers

A. Inbound mail is allowed to a gateway host only (port 25 is for SMTP incoming)

B. explicit statement of the default policy

C. tries to specify that any inside host can send mail to the outside, but has problem that an outside machine could be configured to have some other application linked to port 25

D. properly implements mail sending rule, by checking ACK flag of a TCP segment is set

E. this rule set is one approach to handling FTP connections

Some of the attacks that can be made on packet-filtering routers & countermeasures are:

- **IP address spoofing**: where intruder transmits packets from the outside with internal host source IP addresses, need to filter & discard such packets
- **Source routing attacks**: where source specifies the route that a packet should take to bypass security measures, should discard all source routed packets
- **Tiny fragment attacks**: intruder uses the IP fragmentation option to create extremely small fragments and force the TCP header information into separate fragments to circumvent filtering rules needing full header info,

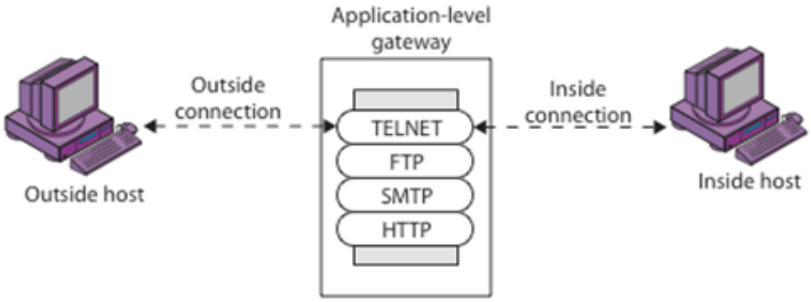
can enforce minimum fragment size to include full header.

Stateful Packet Filters

A traditional packet filter makes filtering decisions on an individual packet basis and does not take into consideration any higher layer context. A stateful inspection packet filter tightens up the rules for TCP traffic by creating a directory of outbound TCP connections, and will allow incoming traffic to high-numbered ports only for those packets that fit the profile of one of the entries in this directory. Hence they are better able to detect bogus packets sent out of context.

Application level gateway

An application-level gateway (or proxy server), acts as a relay of application-level traffic. The user contacts the gateway using a TCP/IP application, such as Telnet or FTP, and the gateway asks the user for the name of the remote host to be accessed. When the user responds and provides a valid user ID and authentication information, the gateway contacts the application on the remote host and relays TCP segments containing the application data between the two endpoints. If the gateway does not implement the proxy code for a specific application, the service is not supported and cannot be forwarded across the firewall.

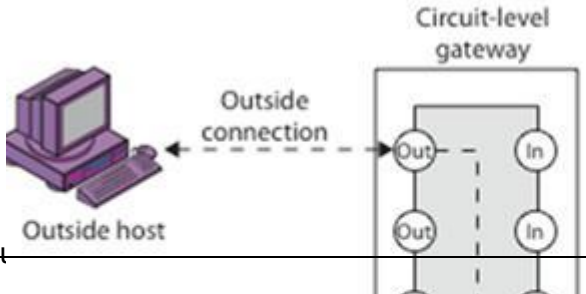


(b) Application-level gateway

Application-level gateways tend to be more secure than packet filters. Rather than trying to deal with the numerous possible combinations that are to be allowed and forbidden at the TCP and IP level, the application-level gateway need only scrutinize a few allowable applications. In addition, it is easy to log&audit all incoming traffic at the application level. A prime disadvantage of this type of gateway is the additional processing overhead on each connection. In effect, there are 2 spliced connections between the end users, with the gateway at the splice point, and the gateway must examine and forward all traffic in both directions.

Circuit Level Gateway

A circuit-level gateway relays two TCP connections, one between itself and an inside TCP user, and the other between itself and a TCP user on an outside host. Once the two connections are established, it relays TCP data from one connection to the other without examining its contents. The security function consists of determining which connections will be allowed. It is typically used when internal users are trusted to decide what external services to access.



One of the most common circuit-level gateways is SOCKS, defined in RFC 1928. It consists of a SOCKS server on the firewall, and a SOCKS library & SOCKS-aware applications on internal clients. The protocol described here is designed to provide a framework for client-server applications in both the TCP and UDP domains to conveniently and securely use the services of a network firewall. The protocol is conceptually a "shim-layer" between the application layer and the transport layer, and as such does not provide network-layer gateway services, such as forwarding of ICMP messages.

Bastion Host

A bastion host is a critical strong point in the network's security, serving as a platform for an application-level or circuit-level gateway, or for external services. It is thus potentially exposed to "hostile" elements and must be secured to withstand this. Common characteristics of a bastion host include that it:

- executes a secure version of its O/S, making it a trusted system
- has only essential services installed on the bastion host
- may require additional authentication before a user is allowed access to the proxy services
- is configured to support only a subset of the standard application's command set, with access only to specific hosts
- maintains detailed audit information by logging all traffic
- has each proxy module a very small software package specifically designed for network security
- has each proxy independent of other proxies on the bastion host
- have a proxy performs no disk access other than to read its initial configuration file
- have each proxy run as a non-privileged user in a private and secured directory

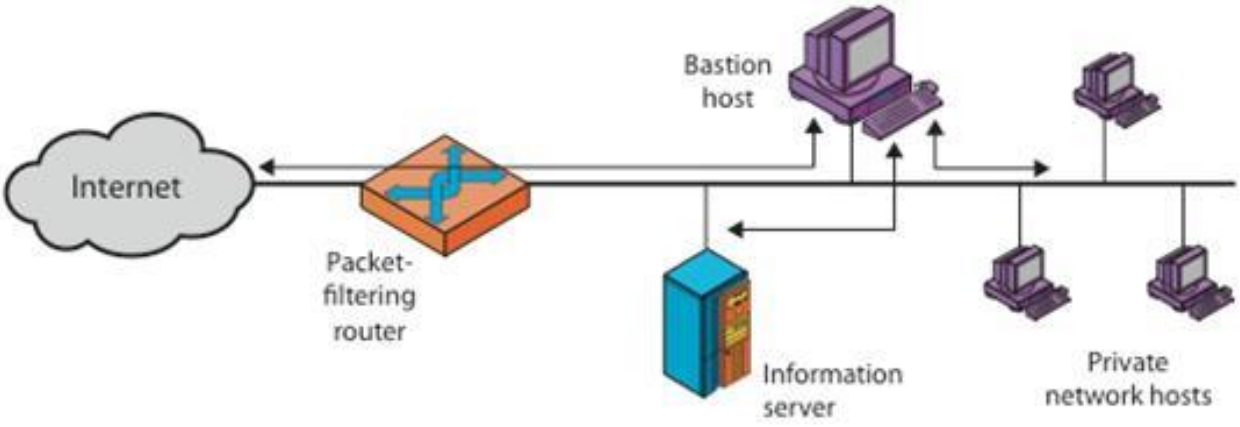
A bastion host may have two or more network interfaces (or ports), and must be trusted to enforce trusted separation between these network connections, relaying traffic only according to policy.

Firewall Configurations

In addition to the use of a simple configuration consisting of a single system, more complex configurations are possible and indeed more common. There are three common firewall configurations.

The following figure shows the "**screened host firewall, single-homed bastion configuration**", where the firewall consists of two systems:

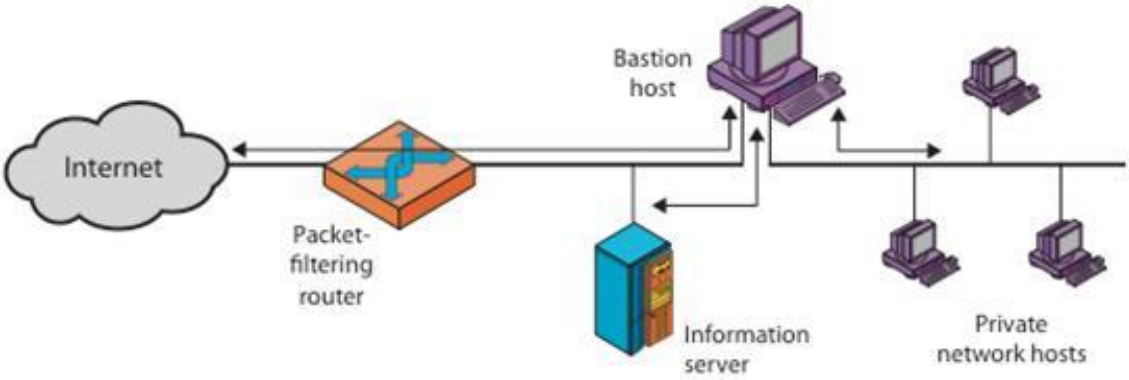
- a packet-filtering router - allows Internet packets to/from bastion only
- a bastion host - performs authentication and proxy functions



(a) Screened host firewall system (single-homed bastion host)

This configuration has greater security, as it implements both packet-level & application-level filtering, forces an intruder to generally penetrate two separate systems to compromise internal security, & also affords flexibility in providing direct Internet access to specific internal servers (eg web) if desired.

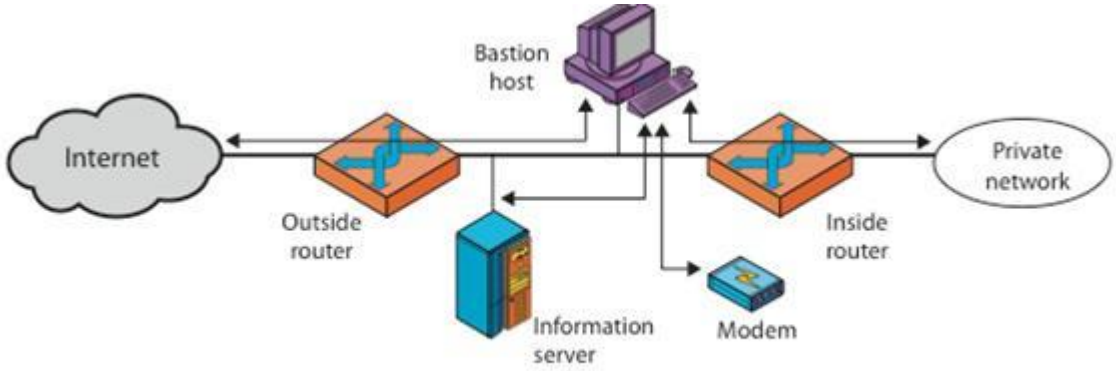
The next configuration illustrates the "**screened host firewall, dual-homed bastion configuration**" which physically separates the external and internal networks, ensuring two systems must be compromised to breach security. The advantages of dual layers of security are also present here.



(b) Screened host firewall system (dual-homed bastion host)

Again, an information server or other hosts can be allowed direct communication with the router if this is in accord with the security policy, but are now separated from the internal network.

The third configurations illustrated below shows the "screened subnet firewall configuration", being the most secure shown.



(c) Screened-subnet firewall system

It has two packet-filtering routers, one between the bastion host and the Internet and the other between the bastion host and the internal network, creating an isolated sub-network. This may consist of simply the bastion host but may also include one or more information servers and modems for dial-in capability. Typically, both the Internet and the internal network have access to hosts on the screened subnet, but traffic across the screened subnet is blocked.

This configuration offers several advantages:

• There are now three levels of defense to thwart intruders

• The outside router advertises only the existence of the screened subnet to the Internet; therefore the internal network is invisible to the Internet

• Similarly, the inside router advertises only the existence of the screened subnet to the internal network; hence systems on the inside network cannot construct direct routes to the Internet

Trusted Systems

Data Access Control

A successful logon would not be sufficient for a system to grant access if it includes sensitive information in its data base. A user can be identified to the system by user access control procedure, where each user is associated with a profile that specifies permissible operations and file accesses enabling the operating system to enforce them. A general model of access control is that of an access matrix, the basic elements of which are:

• *Subject*: An entity (typically a process) capable of accessing objects

• *Object*: Anything to which access is controlled, eg files, portions of files, programs, memory segments

• *Access right*: The way in which an object is accessed by a subject, eg. read, write and execute

One axis of an access matrix consists of identified subjects that may attempt data access, the other lists objects that may be accessed, & each entry in the matrix indicates the access rights of that subject for that object.

	Program1	. . .	SegmentA	SegmentB
Process1	Read Execute		Read Write	
Process2				Read
. . .				

(a) Access matrix

In practice, an access matrix is usually sparse and is implemented by decomposition in one of two ways. If decomposed by columns, you have **access control lists**, which list users & their permitted access rights for each object. If decomposed by rows it yields**capability tickets**, which specify authorized objects & operations for a user. These tickets must be unforgivable which is made possible by having the operating system hold all the tickets on behalf of users and hold them in a region of memory, inaccessible to users.

Access Control List for Program1:
Process1 (Read, Execute)
Access Control List for SegmentA:
Process1 (Read, Write)
Access Control List for SegmentB:
Process2 (Read)

Access Control List

Capability List for Process1:
Program1 (Read, Execute)
SegmentA (Read, Write)
Capability List for Process2:
SegmentB (Read)

Capability List

Concept of Trusted Systems

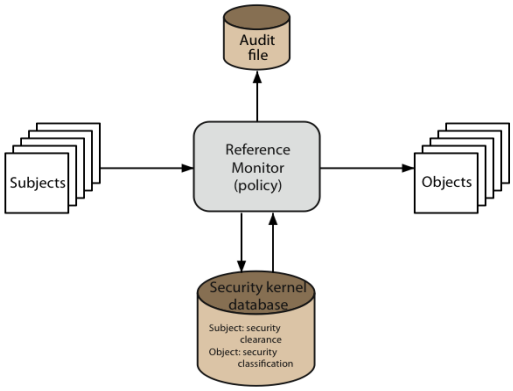
A widely applicable approach for protection of data and resources is based on levels of security. This is commonly found in military, where information is categorized as unclassified (U), confidential (C), secret (S), top secret (TS), or beyond. This concept is equally applicable in other areas, where information can be organized into categories and users can be granted clearances to access certain categories of data. When multiple categories or levels of data are defined, the requirement is referred to as **multilevel security**. The general statement of the requirement for multilevel security is that a subject at a high level may not convey information to a subject at a lower or non-comparable level unless that flow accurately reflects the will of an authorized user. For implementation purposes, this requirement is in two parts and is simply stated. A multilevel secure system must enforce the following:

- **No read-up**: A subject can only read an object of less or equal security level. This is referred to in the literature as the **simple security property**
- **No write-down**: A subject can write into an object of greater or equal security level. This is referred to as the ***-property** (pronounced star property)

A combination of hardware, software, and firmware that implements the Reference Monitor concept is called the *Reference Validation Mechanism* and has the following properties:

- *Complete mediation*: The Reference Validation Mechanism must always be invoked.
- *Isolation:* The Reference Validation Mechanism must be tamperproof.
- *Verifiability:* The Reference Validation Mechanism must be small enough to be subjected to analysis and tests to ensure that it is correct.

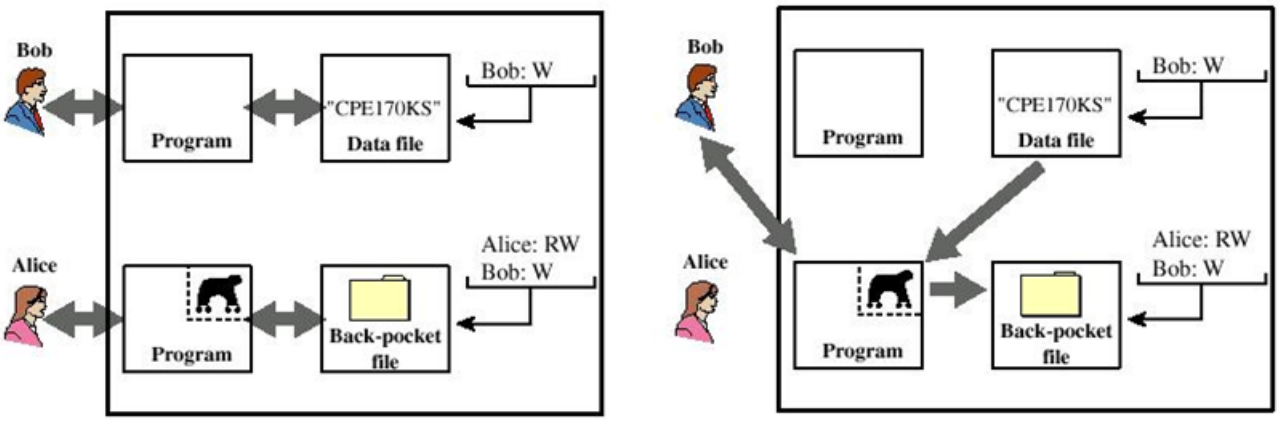
The above mentioned requirements are very stiff. Complete mediation requires that every access to data within main memory and on disk and tape must be mediated. Though pure software implementation is not practical, solution is at least partly hardware implementation. The requirement for isolation means that it must not be possible for an attacker, no matter how clever, to change the logic of the reference monitor or the contents of the security kernel database. Finally, the requirement for mathematical proof is formidable for something as complex as a general-purpose computer. A system that can provide such verification is referred to as a **trusted system**.



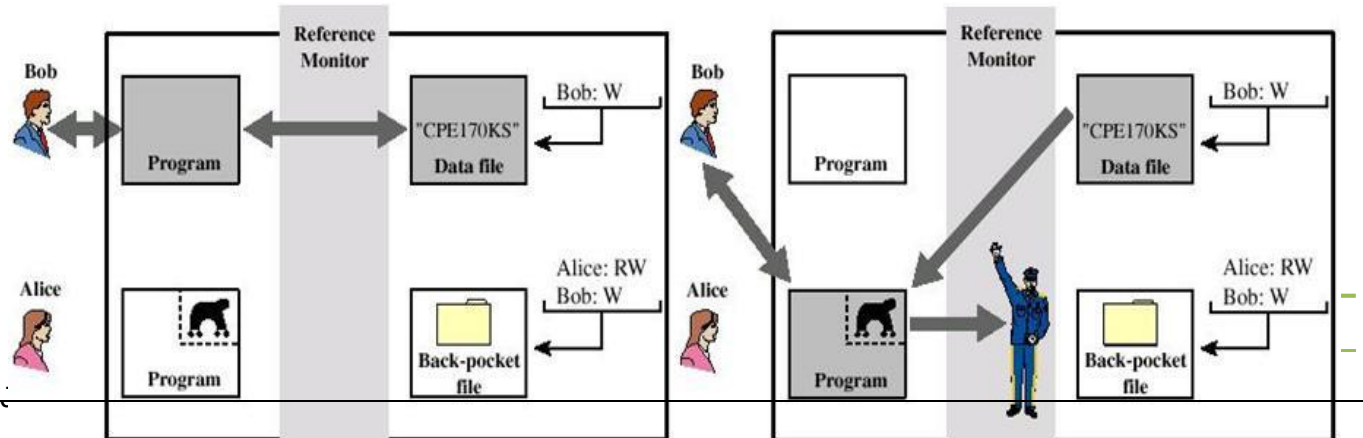
A final element in the Reference Monitor concept is an audit file. Important security events, such as detected security violations and authorized changes to the security kernel database, are stored in audit file.

Trojan horse Defence

A way of securing against Trojan horse attacks is the use of a secure, trusted operating system.



In the above example, a Trojan horse is used to get around the access control list, which is the standard security mechanism. Consider a user Bob interacts through a program with a data file containing the critically sensitive character string “CPE170KS”. He has created the file such that only the processes that are owned by Bob my access the file i.e. (read or write). A malicious user Alice gains legitimate access to the system and installs a Trojan horse program and a private file named as “back pocket”. Alice gives read/write permissions to himself, but write- only permission to Bob. Alice induces Bob to invoke the Trojan horse program, which detects Bob’s execution and copies the sensitive character string into the Alice’s back pocket file. Both read and write satisfy the constraints of the access control lists. Alice has access to Bob’s file at a later time. Using a secure operating system has the following scenario…



Intrusion Detection

Intruders: A significant security problem for networked systems is hostile, or at least unwanted, trespass being unauthorized login or use of a system, by local or remote users; or by software such as a virus, worm, or Trojan horse.

One of the two most publicized threats to security is the intruder (or hacker or cracker), which

Anderson identified three classes of:

- Masquerader: An individual who is not authorized to use the computer (outsider)
- Misfeasor: A legitimate user who accesses unauthorized data, programs, or resources (insider)
- Clandestine user: An individual who seizes supervisory control of the system and uses this control to evade auditing and access controls or to suppress audit collection (either) Intruder attacks range from the benign (simply exploring net to see what is there); to the serious (who attempt to read privileged data, perform unauthorized modifications, or disrupt system). One of the results of the growing awareness of the intruder problem has been the establishment of a number of computer emergency response teams (CERTs). These cooperative ventures collect information about system vulnerabilities and disseminate it to systems managers. The techniques and behavior patterns of intruders are constantly shifting, to exploit newly discovered weaknesses and to evade detection and countermeasures. Even so, intruders typically follow one of a number of recognizable behavior patterns, and these patterns typically differ from those of ordinary users.

The following lists the following examples of intrusion:

•Performing a remote root compromise of an e-mail server

•Defacing a Web server

•Guessing and cracking passwords

•Copying a database containing credit card numbers

•Viewing sensitive data, including payroll records and medical information, without authorization

•Running a packet sniffer on a workstation to capture usernames and passwords

•Using a permission error on an anonymous FTP server to distribute pirated software and music files

•Dialing into an unsecured modem and gaining internal network access

•Posing as an executive, calling the help desk, resetting the executive's e-mail password, and learning the new password

•Using an unattended, logged-in workstation without permission

Hackers: Traditionally, those who hack into computers do so for the thrill of it or for status. The hacking community is a strong meritocracy in which status is determined by level of competence. Thus, attackers often look for targets of opportunity, and then share the information with others. Benign intruders might be tolerable, although they do consume resources and may slow performance for legitimate users. However, there is no way in advance to know whether an intruder will be benign or malign. Consequently, even for systems with no

particularly sensitive resources, there is a motivation to control this problem.

Intrusion detection systems (IDSs) and intrusion prevention systems (IPSs) are designed to counter this type of hacker threat. In addition to using such systems, organizations can consider restricting remote logons to specific IP addresses and/or use virtual private network technology. Unfortunately, hackers can also gain access to CERT reports. Thus, it is important for system administrators to quickly insert all software patches to discovered vulnerabilities. Examples of Hackers behavior

1. select target using IP lookup tools
2. map network for accessible services
3. identify potentially vulnerable services
4. brute force (guess) passwords
5. install remote administration tool
6. wait for admin to log on and capture password
7. use password to access remainder of network

Insider Attacks: Insider attacks are among the most difficult to detect and prevent. Employees already have access and knowledge about the structure and content of corporate databases. Insider attacks can be motivated by revenge of simply a feeling of entitlement. Examples of Insider Behavior are:

1. create network accounts for themselves and their friends
2. access accounts and applications they wouldn't normally use for their daily jobs
3. e-mail former and prospective employers
4. conduct furtive instant-messaging chats
5. visit web sites that cater to disgruntled employees, such as f'dcompany.com
6. perform large downloads and file copying
7. access the network during off hours

The objective of the intruder is to gain access to a system or to increase the range of privileges accessible on a system. Most initial attacks use system or software vulnerabilities that allow a user to execute code that opens a back door into the system. Alternatively, the intruder attempts to acquire information that should have been protected. In some cases, this information is in the form of a user password. With knowledge of some other user's password, an intruder can log in to a system and exercise all the privileges accorded to the legitimate user. Knowing the standard attack methods is a key element in limiting your vulnerability. The basic aim is to gain access and/or increase privileges on some system.

Password guessing is a common attack. If an attacker has obtained a poorly protected password file, then can mount attack off-line, so target is unaware of its progress. Some O/S take less care than others with their password files. If have to actually attempt to login to check guesses, then system should detect an abnormal number of failed logins, and hence trigger appropriate countermeasures by admins/security. Likelihood of success depends very much on how well the passwords are chosen. Unfortunately, users often don’t choose.

There is also a range of ways of "capturing" a login/password pair, from the low-tech looking over the shoulder, to the use of Trojan Horse programs (eg. game program or nifty utility with a covert function as well as the overt behaviour), to sophisticated network monitoring tools, or extracting recorded info after a successful login - say from web history or cache, or last number dialled memory on phones etc. Need to educate users to be aware of whose around, to check they really are interacting with the computer system (trusted path), to beware of unknown source s/w, to use secure network connections (HTTPS, SSH, SSL), to flush browser/phone histories

after use etc.

Approaches to Intrusion Detection

Can identify the following approaches to intrusion detection:

1. **Statistical anomaly detection**: collect data relating to the behavior of legitimate users, then use statistical tests to determine with a high level of confidence whether new behavior is legitimate user behavior or not.

a. Threshold detection: define thresholds, independent of user, for the frequency of occurrence of events.
b. Profile based: develop profile of activity of each user and use to detect changes in the behavior.

2. **Rule-based detection**: attempt to define a set of rules used to decide if given behavior is an intruder

a. Anomaly detection: rules detect deviation from previous usage patterns

b. Penetration identification: expert system approach that searches for suspicious behavior

In a nutshell, statistical approaches attempt to define normal, or expected, behavior, whereas rule-based approaches attempt to define proper behavior. In terms of the types of attackers listed earlier, statistical anomaly detection is effective against masqueraders, who are unlikely to mimic the behavior patterns of the accounts they appropriate. On the other hand, such techniques may be unable to deal with misfeasors. For such attacks, rule-based approaches may be able to recognize events and sequences that, in context, reveal penetration. In practice, a system may exhibit a combination of both approaches to be effective against a broad range of attacks.

Audit Records

A fundamental tool for intrusion detection is the audit record. Some record of ongoing activity by users must be maintained as input to an intrusion detection system. Basically, two plans are used:

- **Native audit records**: Virtually all main O/S's include accounting software that collects information on user activity, advantage is its already there, disadvantage is it may not contain the needed information.
- **Detection-specific audit records**: implement collection facility to generates custom audit records with desired info, advantage is it can be vendor independent & portable, disadvantage is extra overhead involved.

Statistical Anomaly Detection

Statistical anomaly detection techniques fall into two broad categories: threshold detection and profile-based systems. Threshold detection involves counting the number of occurrences of a specific event type over an interval of time. If the count surpasses what is considered a reasonable number that one might expect to occur, then intrusion is assumed. By itself, is a crude and ineffective detector of even moderately sophisticated attacks? Profile-based anomaly detection focuses on characterizing past behavior of users or groups, and then detecting significant deviations. A profile may consist of a set of parameters, so that deviation on just a single parameter may not be sufficient in itself to signal an alert. Foundation of this approach is analysis of audit records. . Examples of metrics that are useful for profile-based intrusion detection are: counter, gauge, interval timer, resource use. Given these general metrics, various tests can be performed to determine whether current activity fits within acceptable limits, such as: Mean and standard deviation, Multivariate, Markov process, Time series, Operational. The main advantage of the use of statistical profiles is that a prior knowledge of security flaws is not required. Thus it should be readily portable among a variety of systems.

Rule Based Intrusion Detection

Rule-based techniques detect intrusion by observing events in the system and applying a set of rules that lead to a decision regarding whether a given pattern of activity is or is not suspicious. Can characterize approaches as either anomaly detection or penetrate identification, although there is overlap. Rule-based anomaly detection is

similar in terms of its approach and strengths to statistical anomaly detection. Historical audit records are analyzed to identify usage patterns and to automatically generate rules that describe those patterns. Current behavior is then observed and matched against the set of rules to see if it conforms to any historically observed pattern of behavior. As with statistical anomaly detection, rule-based anomaly detection does not require knowledge of security vulnerabilities within the system.

Rule-based penetration identification takes a very different approach based on expert system technology. It uses rules for identifying known penetrations or penetrations that would exploit known weaknesses, or identify suspicious behavior. The rules used are specific to machine and operating system. The rules are generated by "experts", from interviews of system administrators and security analysts. Thus the strength of the approach depends on the skill of those involved in setting up the rules.

Base-Rate Fallacy

To be of practical use, an intrusion detection system should detect a substantial percentage of intrusions while keeping the false alarm rate at an acceptable level. If only a modest percentage of actual intrusions are detected, the system provides a false sense of security. On the other hand, if the system frequently triggers an alert when there is no intrusion (a false alarm), then either system managers will begin to ignore the alarms, or much time will be wasted analyzing the false alarms. Unfortunately, because of the nature of the probabilities involved, it is very difficult to meet the standard of high rate of detections with a low rate of false alarms. A study of existing intrusion detection systems indicated that current systems have not overcome the problem of the base-rate fallacy.

Distributed Intrusion Detection

Until recently, work on intrusion detection systems focused on single-system standalone facilities. The typical organization, however, needs to defend a distributed collection of hosts supported by a LAN or internetwork, where a more effective defense can be achieved by coordination and cooperation among intrusion detection systems across the network.

Porras points out the following major issues in the design of a distributed IDS:

- A distributed intrusion detection system may need to deal with different audit record formats
- One or more nodes in the network will serve as collection and analysis points for the data, which must be securely transmitted to them
- Either a centralized (single point, easier but bottleneck) or decentralized (multiple centers must coordinate) architecture can be used.

Honeypots

Honeypots are decoy systems, designed to lure a potential attacker away from critical systems, and:

- divert an attacker from accessing critical systems
- collect information about the attacker's activity
- encourage the attacker to stay on the system long enough for administrators to respond

Password Management:

The front line of defense against intruders is the password system, where a user provides a name/login identifier (ID) and a password. The password serves to authenticate the ID of the individual logging on to the system. Passwords are usually stored encrypted rather than in the clear (which would make them more

vulnerable to theft). Unix systems traditionally used a multiple DES variant with salt as a one-way hash function (see text). More recent Operating systems use a cryptographic hash function (eg. MD5). The file containing these passwords hashes needs access control protections to make guessing attacks harder.

Goal is to eliminate guessable passwords while allowing user to select a memorable password. Four basic techniques are in use: education, computer generation, reactive checking & proactive checking.

The user education strategy tells users the importance of using hard-to-guess passwords and provides guidelines for selecting strong passwords, but it needs their cooperation. The problem is that many users will simply ignore the guidelines.

Computer-generated passwords create a password for the user, but have problems. If the passwords are quite random in nature, users will not be able to remember them. Even if the password is pronounceable, the user may have difficulty remembering it and so be tempted to write it down. In general, computer-generated password schemes have a history of poor acceptance by users. FIPS PUB 181 defines one of the best-designed automated password generators. The standard includes not only a description of the approach but also a complete listing of the C source code of the algorithm, which generates words by forming a random set of pronounceable syllables and concatenating them to form a word.

A reactive password checking strategy is one in which the system periodically runs its own password cracker to find guessable passwords. The system cancels any passwords that are guessed and notifies the user. Drawbacks are that it is resource intensive if the job is done right, and any existing passwords remain vulnerable until the reactive password checker finds them.

The most promising approach to improved password security is a proactive password checker, where a user is allowed to select his or her own password, but the system checks to see if it is allowable and rejects it if not. The trick is to strike a balance between user acceptability and strength. The first approach is a simple system for rule enforcement, enforcing say guidelines from user education. This may not be good enough. Another approach is to compile a large dictionary of possible “bad” passwords, and check user passwords against this disapproved list. But this can be very large & slow to search. A third approach is based on rejecting words using either a Markov model of guessable passwords, or a Bloom filter. Both attempt to identify good or bad passwords without keeping large dictionaries.